

PRIXY

Gestion de la Salle Beryl — Application Intranet

CAHIER DES CHARGES TECHNIQUE

Stack PHP MVC — Version mise à jour après avenant N°1

Type de document	CDC Technique
Version	V2.0
Date	Mars 2026
Projet	Application Intranet — Salle Beryl — PRIXY
Statut	Document officiel

BTS SIO — Option SLAM — Session 2025-2026

Ce document remplace le CDC Technique V1.0. Il intègre les modifications de l'Avenant N°1 (suppression de React et de l'API REST).

1. Objectif du document

Ce document définit les exigences techniques du projet PRIXY après l'avenant N°1. Il s'adresse à l'équipe de développement et aux administrateurs système.

Par rapport à la V1.0, les seuls changements concernent la stack technique : suppression de React, des frameworks JS complexes et de l'API REST. Tout le reste est identique.

1.1 Technologies obligatoires

Couche	Technologie retenue	Justification / version minimum
Frontend	HTML5 + CSS3 + JavaScript ES6+	Pas de framework JS — Vanilla uniquement — Bootstrap 5 pour le responsive
Framework CSS	Bootstrap 5.x	Grille responsive — Composants prêts à l'emploi
Backend	PHP 8.1+	Architecture MVC — Rendu HTML côté serveur — PDO obligatoire — Composer
Base de données	MySQL 8.0+	InnoDB — utf8mb4 — Inchangé par rapport à la V1.0
Serveur web	Apache 2.4	HTTPS obligatoire — mod_rewrite activé
Versioning	Git (GitHub ou GitLab)	Branches : main, develop, feature/* — Pull Request avant merge
Email	PHPMailer 6.x	SMTP interne PRIXY — Templates HTML
Tests	PHPUnit 10.x	Couverture cible : 70% minimum du code PHP

1.2 Technologies supprimées (par rapport à la V1.0)

Technologie retirée	Pourquoi	Remplacée par
React / Alpine.js / Vue.js	Complexité inutile pour ce projet	JavaScript vanilla ES6+ uniquement
API REST (endpoints JSON + JWT)	Surcharge architecturale non nécessaire	Contrôleurs PHP classiques — Rendu HTML côté serveur

2. Architecture de l'application

L'application suit une architecture MVC PHP classique. Le serveur PHP génère les pages HTML complètes et les renvoie au navigateur. Le JavaScript est utilisé uniquement pour l'interactivité côté client (affichage dynamique, validations de formulaire, etc.) sans communication asynchrone complexe.

Couche	Technologies	Rôle
Vue (View)	HTML5 + CSS3 + JavaScript ES6+ + Bootstrap 5	Ce que l'utilisateur voit — Généré par PHP côté serveur
Contrôleur (Controller)	PHP 8.1+ — MVC	Reçoit les requêtes, appelle les modèles, renvoie les vues HTML
Modèle (Model)	PHP + PDO + MySQL 8.0	Lecture et écriture en base de données
Routeur	PHP — .htaccess Apache	Redirige les URLs vers les bons contrôleurs
Notifications email	PHPMailer + SMTP interne	Envoi automatique depuis les contrôleurs PHP

2.1 Comment fonctionne la navigation

Chaque action de l'utilisateur (clic sur un bouton, soumission d'un formulaire) envoie une requête HTTP au serveur Apache. PHP traite la requête, interroge MySQL si besoin, et renvoie une page HTML complète. Aucun appel JSON/AJAX complexe — on reste sur du PHP classique.

Exception : quelques interactions légères en JavaScript vanilla (mise à jour dynamique d'un créneau, validation de formulaire en temps réel) peuvent être utilisées ponctuellement sans framework.

3. Prérequis serveur de production

- Système : Ubuntu Server 22.04 LTS — RAM minimum 4 Go — Disque SSD 50 Go
- PHP 8.1+ avec les extensions : PDO, pdo_mysql, mbstring, openssl, json, ldap, curl
- MySQL 8.0 — Accès local uniquement — Compte applicatif dédié (pas root)
- Apache 2.4 avec mod_rewrite et mod_ssl — HTTPS obligatoire
- Certificat SSL valide (auto-signé accepté en intranet)
- Composer 2.x — Git
- Accès SMTP vers le serveur de messagerie interne PRIXY

4. Exigences de sécurité

Les exigences de sécurité sont inchangées par rapport au CDC V1.0.

Catégorie	Obligation	Comment l'implémenter
Mots de passe	Hash bcrypt uniquement	PASSWORD_BCRYPT — Coût ≥ 12 — Jamais de MD5/SHA1
Sessions	Sécuriser les cookies	HttpOnly + Secure + SameSite=Strict — Régénérer l'ID à chaque connexion
Requêtes SQL	Requêtes préparées	PDO::prepare() exclusivement — Jamais de variable collée dans le SQL
Affichage HTML	Échapper les données	htmlspecialchars() sur toute variable affichée
Formulaires	Token CSRF	Token unique par formulaire — Vérification côté serveur
Fichiers uploadés	Valider les fichiers	Vérifier le vrai type MIME — Renommer — Stocker hors du dossier web
Transport	HTTPS	TLS 1.2 minimum — Rediriger HTTP $\rightarrow$ HTTPS — Header HSTS
Droits	Vérification serveur	Vérifier rôle + permission à chaque contrôleur PHP

5. Tests à réaliser

Les exigences de tests sont inchangées par rapport au CDC V1.0.

Type de test	Outil recommandé	Ce qu'il faut couvrir
Tests unitaires	PHPUnit 10.x	70% des classes PHP — 100% des règles métier RG-01 à RG-10
Tests fonctionnels	Manuel + Selenium	Connexion, création de réservation, annulation, workflow d'approbation externe
Tests de sécurité	OWASP ZAP + Manuel	Injection SQL, XSS, CSRF, accès non autorisés
Tests de performance	Apache JMeter	30 utilisateurs simultanés — Temps de réponse < 2 secondes
Tests de compatibilité	DevTools navigateur	Chrome 120+, Firefox 120+, Edge 120+

6. Conformité RGPD

Les obligations RGPD sont inchangées. Voir le Registre RGPD (document 10) pour le détail.

Obligation	Ce qu'on doit faire
Minimisation des données	Pas de champ inutile dans les formulaires
Durée de conservation	Réservations : 3 ans — Logs : 1 an — Suppression automatique via tâche planifiée
Droits des utilisateurs	Consultation, rectification et suppression des données accessibles depuis l'interface
Sécurité des données	Mots de passe chiffrés — HTTPS — Sauvegardes sécurisées

7. Signatures

Ce CDC Technique V2.0 remplace et annule le CDC Technique V1.0 sur la partie stack technique uniquement. Il est signé conjointement avec l'Avenant N°1.

Pour PRIXY — Commanditaire	Chef de projet — Équipe SLAM
Nom et prénom : _____	Nom et prénom : Mael DAVIDENKO
Qualité : _____	Rôle : Chef de projet
Date : _____	Date : 16/03/2026
Lu et approuvé — Signature :	Lu et approuvé — Signature :
	